SQL JOIN LAB RECORD

Name: Rakshit Rameshbabu
Register Number: 24BYB1117
Course:BCSE302P Database Systems Lab
Experiment Title: Implementation of SQL JOIN Operations
Date:24-02-2026

A. Table Creation

1. Create table CUSTOMER1_24BYB1117 with attributes:
   - CUST_ID (Primary Key)
   - CUST_NAME
   - CITY

```
SQL> CREATE TABLE CUSTOMER1_24BYB1117 (
  2  CUST_ID NUMBER PRIMARY KEY,
  3  CUST_NAME VARCHAR2(50),
  4  CITY VARCHAR2(50)
  5  );

Table created.
```

2. Create table ORDERS_24BYB1117 with attributes:
   - ORDER_ID (Primary Key)
   - CUST_ID

- ORDER_DATE
- AMOUNT

```
SQL> CREATE TABLE ORDERS_24BYB1117 (
  2    ORDER_ID NUMBER PRIMARY KEY,
  3    CUST_ID NUMBER,
  4    ORDER_DATE DATE,
  5    AMOUNT NUMBER
  6  );

Table created.
```

3. Create table PRODUCT_24BYB1117 with attributes:
   - PRODUCT_ID (Primary Key)
   - PRODUCT_NAME
   - CATEGORY

```
SQL>
SQL> CREATE TABLE PRODUCT_24BYB1117 (
  2    PRODUCT_ID NUMBER PRIMARY KEY,
  3    PRODUCT_NAME VARCHAR2(50),
  4    CATEGORY VARCHAR2(50)
  5  );

Table created.
```

4. Create table ORDERITEM_24BYB1117 with attributes:
   - ORDER_ID

- PRODUCT_ID
- QUANTITY

```
SQL> CREATE TABLE ORDERITEM_24BYB1117 (
  2  ORDER_ID NUMBER,
  3  PRODUCT_ID NUMBER,
  4  QUANTITY NUMBER
  5  );

Table created.
```

B. Insert Multiple Records

5. Insert multiple records into CUSTOMER1_24BYB1117.
6. Insert multiple records into ORDERS_24BYB1117.
7. Insert multiple records into PRODUCT_24BYB1117.
8. Insert multiple records into ORDERITEM_24BYB1117.

```
SQL> INSERT ALL
  2  INTO CUSTOMER1_24BYB1117 VALUES (1,'Arun','Chennai')
  3  INTO CUSTOMER1_24BYB1117 VALUES (2,'Divya','Bangalore')
  4  INTO CUSTOMER1_24BYB1117 VALUES (3,'Karthik','Hyderabad')
  5  INTO CUSTOMER1_24BYB1117 VALUES (4,'Meena','Delhi')
  6  INTO CUSTOMER1_24BYB1117 VALUES (5,'Rahul','Mumbai')
  7  SELECT * FROM DUAL;

5 rows created.

SQL> INSERT ALL
  2  INTO ORDERS_24BYB1117 VALUES (101,1,DATE '2024-01-10',5000)
  3  INTO ORDERS_24BYB1117 VALUES (102,2,DATE '2024-01-11',7000)
  4  INTO ORDERS_24BYB1117 VALUES (103,1,DATE '2024-01-15',2000)
  5  INTO ORDERS_24BYB1117 VALUES (104,3,DATE '2024-01-20',9000)
  6  INTO ORDERS_24BYB1117 VALUES (105,5,DATE '2024-01-25',3000)
  7  SELECT * FROM DUAL;

5 rows created.

SQL> INSERT ALL
  2  INTO PRODUCT_24BYB1117 VALUES (201,'Laptop','Electronics')
  3  INTO PRODUCT_24BYB1117 VALUES (202,'Mouse','Electronics')
  4  INTO PRODUCT_24BYB1117 VALUES (203,'Chair','Furniture')
  5  INTO PRODUCT_24BYB1117 VALUES (204,'Table','Furniture')
  6  INTO PRODUCT_24BYB1117 VALUES (205,'Headphones','Electronics')
  7  SELECT * FROM DUAL;

5 rows created.

SQL> INSERT ALL
  2  INTO ORDERITEM_24BYB1117 VALUES (101,201,1)
  3  INTO ORDERITEM_24BYB1117 VALUES (101,202,2)
  4  INTO ORDERITEM_24BYB1117 VALUES (102,203,1)
  5  INTO ORDERITEM_24BYB1117 VALUES (103,205,1)
  6  INTO ORDERITEM_24BYB1117 VALUES (104,204,1)
  7  SELECT * FROM DUAL;

5 rows created.
```

Practice Questions

Natural Join

Q7: Display all customer names and their order amounts using NATURAL JOIN.

Q8: Show product names and quantities sold.

Q9: List order IDs with product categories.

Q10: Display customers who placed more than one order.

Q11: Show orders with electronics items.

Q12: Display all furniture orders.

```
SQL> -- Q7: Display all customer names and their order amounts using NATURAL JOIN.
SQL> SELECT CUST_NAME, AMOUNT
  2  FROM CUSTOMER1_24BYB1117
  3  NATURAL JOIN ORDERS_24BYB1117;

CUST_NAME       AMOUNT
=============== ======
Arun              5000
Divya             7000
Arun              2000
Karthik           9000
Rahul             3000
SQL>
SQL> -- Q8: Show product names and quantities sold.
SQL> SELECT PRODUCT_NAME, QUANTITY
  2  FROM PRODUCT_24BYB1117
  3  NATURAL JOIN ORDERITEM_24BYB1117;

PRODUCT_NAME    QUANTITY
=============== ========
Laptop                 1
Mouse                  2
Chair                  1
Headphones             1
Table                  1
SQL>
SQL> -- Q9: List order IDs with product categories.
SQL> SELECT ORDER_ID, CATEGORY
  2  FROM PRODUCT_24BYB1117
  3  NATURAL JOIN ORDERITEM_24BYB1117;

ORDER_ID CATEGORY
======== ===============
     101 Electronics
     101 Electronics
     102 Furniture
     103 Electronics
     104 Furniture
```

```
SQL> -- Q10: Display customers who placed more than one order.
SQL> SELECT CUST_NAME
  2  FROM CUSTOMER1_24BYB1117
  3  NATURAL JOIN ORDERS_24BYB1117
  4  GROUP BY CUST_NAME
  5  HAVING COUNT(*) > 1;

CUST_NAME
===============
Arun
SQL>
SQL> -- Q11: Show orders with electronics items.
SQL> SELECT ORDER_ID, PRODUCT_NAME
  2  FROM PRODUCT_24BYB1117
  3  NATURAL JOIN ORDERITEM_24BYB1117
  4  WHERE CATEGORY='Electronics';

ORDER_ID PRODUCT_NAME
======== ===============
     101 Laptop
     101 Mouse
     103 Headphones
SQL>
SQL> -- Q12: Display all furniture orders.
SQL> SELECT ORDER_ID, PRODUCT_NAME
  2  FROM PRODUCT_24BYB1117
  3  NATURAL JOIN ORDERITEM_24BYB1117
  4  WHERE CATEGORY='Furniture';

ORDER_ID PRODUCT_NAME
======== ===============
     102 Chair
     104 Table
```

Left Outer Join

Q13: List all customers and their order amounts.

Q14: Show customers who never placed an order.

Q15: Display all orders and product names.

Q16: Identify orders without items.

Q17: Display all products and order IDs.

Q18: Show customers with NULL order values.

```
SQL> -- Q13: List all customers and their order amounts.
SQL> SELECT C.CUST_NAME, O.AMOUNT
  2  FROM CUSTOMER1_24BYB1117 C
  3  LEFT JOIN ORDERS_24BYB1117 O
  4  ON C.CUST_ID = O.CUST_ID;

CUST_NAME       AMOUNT
=============== ======
Arun               5000
Divya              7000
Arun               2000
Karthik            9000
Rahul              3000
Meena
SQL>
SQL> -- Q14: Show customers who never placed an order.
SQL> SELECT C.CUST_NAME
  2  FROM CUSTOMER1_24BYB1117 C
  3  LEFT JOIN ORDERS_24BYB1117 O
  4  ON C.CUST_ID = O.CUST_ID
  5  WHERE O.ORDER_ID IS NULL;

CUST_NAME
===============
Meena
SQL>
SQL> -- Q15: Display all orders and product names.
SQL> SELECT O.ORDER_ID, P.PRODUCT_NAME
  2  FROM ORDERS_24BYB1117 O
  3  LEFT JOIN ORDERITEM_24BYB1117 OI
  4  ON O.ORDER_ID = OI.ORDER_ID
  5  LEFT JOIN PRODUCT_24BYB1117 P
  6  ON OI.PRODUCT_ID = P.PRODUCT_ID;

ORDER_ID PRODUCT_NAME
======== ===============
     101 Laptop
     101 Mouse
     102 Chair
     104 Table
     103 Headphones
     105
```

```
SQL> -- Q16: Identify orders without items.
SQL> SELECT O.ORDER_ID
  2  FROM ORDERS_24BYB1117 O
  3  LEFT JOIN ORDERITEM_24BYB1117 OI
  4  ON O.ORDER_ID = OI.ORDER_ID
  5  WHERE OI.PRODUCT_ID IS NULL;

ORDER_ID
========
     105
SQL>
SQL> -- Q17: Display all products and order IDs.
SQL> SELECT P.PRODUCT_NAME, OI.ORDER_ID
  2  FROM PRODUCT_24BYB1117 P
  3  LEFT JOIN ORDERITEM_24BYB1117 OI
  4  ON P.PRODUCT_ID = OI.PRODUCT_ID;

PRODUCT_NAME    ORDER_ID
=============== ========
Laptop               101
Mouse                101
Chair                102
Headphones           103
Table                104
SQL>
SQL> -- Q18: Show customers with NULL order values.
SQL> SELECT C.CUST_NAME
  2  FROM CUSTOMER1_24BYB1117 C
  3  LEFT JOIN ORDERS_24BYB1117 O
  4  ON C.CUST_ID = O.CUST_ID
  5  WHERE O.AMOUNT IS NULL;
```

Right Outer Join

Q19: List all orders with customer names.

Q20: Identify orders with missing customers.

Q21: Display products that were never ordered.

Q22: Show all furniture items and order IDs.

Q23: Display products with NULL orders.

Q24: Show electronics products without orders.

```
SQL> -- Q19: List all orders with customer names.
SQL> SELECT C.CUST_NAME, O.ORDER_ID
  2  FROM CUSTOMER1_24BYB1117 C
  3  RIGHT JOIN ORDERS_24BYB1117 O
  4  ON C.CUST_ID = O.CUST_ID;

CUST_NAME       ORDER_ID
=============== ========
Arun                 103
Arun                 101
Divya                102
Karthik              104
Rahul                105
SQL>
SQL> -- Q20: Identify orders with missing customers.
SQL> SELECT O.ORDER_ID
  2  FROM CUSTOMER1_24BYB1117 C
  3  RIGHT JOIN ORDERS_24BYB1117 O
  4  ON C.CUST_ID = O.CUST_ID
  5  WHERE C.CUST_ID IS NULL;
SQL>
SQL> -- Q21: Display products that were never ordered.
SQL> SELECT P.PRODUCT_NAME
  2  FROM PRODUCT_24BYB1117 P
  3  LEFT JOIN ORDERITEM_24BYB1117 OI
  4  ON P.PRODUCT_ID = OI.PRODUCT_ID
  5  WHERE OI.ORDER_ID IS NULL;
SQL>
SQL> -- Q22: Show all furniture items and order IDs.
SQL> SELECT P.PRODUCT_NAME, OI.ORDER_ID
  2  FROM PRODUCT_24BYB1117 P
  3  RIGHT JOIN ORDERITEM_24BYB1117 OI
  4  ON P.PRODUCT_ID = OI.PRODUCT_ID
  5  WHERE CATEGORY='Furniture';

PRODUCT_NAME    ORDER_ID
=============== ========
Chair                102
Table                104
SQL>
SQL> -- Q23: Display products with NULL orders.
SQL> SELECT P.PRODUCT_NAME
  2  FROM PRODUCT_24BYB1117 P
  3  LEFT JOIN ORDERITEM_24BYB1117 OI
  4  ON P.PRODUCT_ID = OI.PRODUCT_ID
  5  WHERE OI.ORDER_ID IS NULL;
SQL>
SQL> -- Q24: Show electronics products without orders.
SQL> SELECT P.PRODUCT_NAME
  2  FROM PRODUCT_24BYB1117 P
  3  LEFT JOIN ORDERITEM_24BYB1117 OI
  4  ON P.PRODUCT_ID = OI.PRODUCT_ID
  5  WHERE CATEGORY='Electronics'
  6  AND OI.ORDER_ID IS NULL;
```

Full Outer Join

Q25: Show all customers and all orders.

Q26: Identify customers without orders and orders without customers.

Q27: Display all products and order IDs.

Q28: Identify unordered products.

Q29: Show complete customer–order relationship.

Q30: Display unmatched records from both tables.

```
SQL> -- Q25: Show all customers and all orders.
SQL> SELECT C.CUST_NAME, O.ORDER_ID
  2  FROM CUSTOMER1_24BYB1117 C
  3  FULL OUTER JOIN ORDERS_24BYB1117 O
  4  ON C.CUST_ID = O.CUST_ID;

CUST_NAME       ORDER_ID
=============== ========
Arun                 101
Divya                102
Arun                 103
Karthik              104
Rahul                105
Meena
SQL>
SQL> -- Q26: Identify customers without orders and orders without customers.
SQL> SELECT C.CUST_NAME, O.ORDER_ID
  2  FROM CUSTOMER1_24BYB1117 C
  3  FULL OUTER JOIN ORDERS_24BYB1117 O
  4  ON C.CUST_ID = O.CUST_ID
  5  WHERE C.CUST_ID IS NULL
  6  OR O.CUST_ID IS NULL;

CUST_NAME       ORDER_ID
=============== ========
Meena
SQL>
SQL> -- Q27: Display all products and order IDs.
SQL> SELECT P.PRODUCT_NAME, OI.ORDER_ID
  2  FROM PRODUCT_24BYB1117 P
  3  FULL OUTER JOIN ORDERITEM_24BYB1117 OI
  4  ON P.PRODUCT_ID = OI.PRODUCT_ID;

PRODUCT_NAME    ORDER_ID
=============== ========
Laptop               101
Mouse                101
Chair                102
Headphones           103
Table                104
```

```
SQL> -- Q28: Identify unordered products.
SQL> SELECT P.PRODUCT_NAME
  2  FROM PRODUCT_24BYB1117 P
  3  FULL OUTER JOIN ORDERITEM_24BYB1117 OI
  4  ON P.PRODUCT_ID = OI.PRODUCT_ID
  5  WHERE OI.ORDER_ID IS NULL;
SQL>
SQL> -- Q29: Show complete customer-order relationship.
SQL> SELECT *
  2  FROM CUSTOMER1_24BYB1117 C
  3  FULL OUTER JOIN ORDERS_24BYB1117 O
  4  ON C.CUST_ID = O.CUST_ID;

   CUST_ID CUST_NAME       CITY            ORDER_ID    CUST_ID ORDER_DATE   AMOUNT
========== =============== =============== ======== ========== ============ ======
         1 Arun            Chennai              101          1 10-JAN-24      5000
         2 Divya           Bangalore            102          2 11-JAN-24      7000
         1 Arun            Chennai              103          1 15-JAN-24      2000
         3 Karthik         Hyderabad            104          3 20-JAN-24      9000
         5 Rahul           Mumbai               105          5 25-JAN-24      3000
         4 Meena           Delhi
SQL>
SQL> -- Q30: Display unmatched records from both tables.
SQL> SELECT *
  2  FROM CUSTOMER1_24BYB1117 C
  3  FULL OUTER JOIN ORDERS_24BYB1117 O
  4  ON C.CUST_ID = O.CUST_ID
  5  WHERE C.CUST_ID IS NULL
  6  OR O.CUST_ID IS NULL;

   CUST_ID CUST_NAME       CITY            ORDER_ID    CUST_ID ORDER_DATE   AMOUNT
========== =============== =============== ======== ========== ============ ======
         4 Meena           Delhi
```

Cross Join

Q31: Generate all customer–order combinations.

Q32: Generate all customer–product combinations.

Q33: Generate all product–order combinations.

Q34: Count total rows produced by cross join.

Q35: Use WHERE clause to limit cross join output.

Q36: Explain why cross join is risky for large tables.

```
         4 Meena           Delhi
SQL> -- Q31: Generate all customer-order combinations.
SQL> SELECT *
  2  FROM CUSTOMER1_24BYB1117
  3  CROSS JOIN ORDERS_24BYB1117;

   CUST_ID CUST_NAME       CITY            ORDER_ID    CUST_ID ORDER_DATE   AMOUNT
========== ============== ============== ======== ========== ============ ======
         1 Arun           Chennai             101          1 10-JAN-24      5000
         2 Divya          Bangalore           101          1 10-JAN-24      5000
         3 Karthik        Hyderabad           101          1 10-JAN-24      5000
         4 Meena          Delhi               101          1 10-JAN-24      5000
         5 Rahul          Mumbai              101          1 10-JAN-24      5000
         1 Arun           Chennai             102          2 11-JAN-24      7000
         2 Divya          Bangalore           102          2 11-JAN-24      7000
         3 Karthik        Hyderabad           102          2 11-JAN-24      7000
         4 Meena          Delhi               102          2 11-JAN-24      7000
         5 Rahul          Mumbai              102          2 11-JAN-24      7000
         1 Arun           Chennai             103          1 15-JAN-24      2000
         2 Divya          Bangalore           103          1 15-JAN-24      2000
         3 Karthik        Hyderabad           103          1 15-JAN-24      2000
         4 Meena          Delhi               103          1 15-JAN-24      2000
         5 Rahul          Mumbai              103          1 15-JAN-24      2000
         1 Arun           Chennai             104          3 20-JAN-24      9000
         2 Divya          Bangalore           104          3 20-JAN-24      9000
         3 Karthik        Hyderabad           104          3 20-JAN-24      9000
         4 Meena          Delhi               104          3 20-JAN-24      9000
         5 Rahul          Mumbai              104          3 20-JAN-24      9000
         1 Arun           Chennai             105          5 25-JAN-24      3000
         2 Divya          Bangalore           105          5 25-JAN-24      3000
         3 Karthik        Hyderabad           105          5 25-JAN-24      3000
         4 Meena          Delhi               105          5 25-JAN-24      3000
         5 Rahul          Mumbai              105          5 25-JAN-24      3000
SQL>
```

```
SQL> -- Q32: Generate all customer-product combinations.
SQL> SELECT *
  2  FROM CUSTOMER1_24BYB1117
  3  CROSS JOIN PRODUCT_24BYB1117;

   CUST_ID CUST_NAME       CITY            PRODUCT_ID PRODUCT_NAME    CATEGORY
========== ============== ============== ========== ============== ===============
         1 Arun           Chennai                201 Laptop         Electronics
         1 Arun           Chennai                202 Mouse          Electronics
         1 Arun           Chennai                203 Chair          Furniture
         1 Arun           Chennai                204 Table          Furniture
         1 Arun           Chennai                205 Headphones     Electronics
         2 Divya          Bangalore              201 Laptop         Electronics
         2 Divya          Bangalore              202 Mouse          Electronics
         2 Divya          Bangalore              203 Chair          Furniture
         2 Divya          Bangalore              204 Table          Furniture
         2 Divya          Bangalore              205 Headphones     Electronics
         3 Karthik        Hyderabad              201 Laptop         Electronics
         3 Karthik        Hyderabad              202 Mouse          Electronics
         3 Karthik        Hyderabad              203 Chair          Furniture
         3 Karthik        Hyderabad              204 Table          Furniture
         3 Karthik        Hyderabad              205 Headphones     Electronics
         4 Meena          Delhi                  201 Laptop         Electronics
         4 Meena          Delhi                  202 Mouse          Electronics
         4 Meena          Delhi                  203 Chair          Furniture
         4 Meena          Delhi                  204 Table          Furniture
         4 Meena          Delhi                  205 Headphones     Electronics
         5 Rahul          Mumbai                 201 Laptop         Electronics
         5 Rahul          Mumbai                 202 Mouse          Electronics
         5 Rahul          Mumbai                 203 Chair          Furniture
         5 Rahul          Mumbai                 204 Table          Furniture
         5 Rahul          Mumbai                 205 Headphones     Electronics
SQL>
```

```
SQL> -- Q33: Generate all product-order combinations.
SQL> SELECT *
  2  FROM PRODUCT_24BYB1117
  3  CROSS JOIN ORDERS_24BYB1117;

PRODUCT_ID PRODUCT_NAME    CATEGORY        ORDER_ID    CUST_ID ORDER_DATE   AMOUNT
========== =============== =============== ======== ========== ============ ======
       201 Laptop          Electronics          101          1 10-JAN-24      5000
       202 Mouse           Electronics          101          1 10-JAN-24      5000
       203 Chair           Furniture            101          1 10-JAN-24      5000
       204 Table           Furniture            101          1 10-JAN-24      5000
       205 Headphones      Electronics          101          1 10-JAN-24      5000
       201 Laptop          Electronics          102          2 11-JAN-24      7000
       202 Mouse           Electronics          102          2 11-JAN-24      7000
       203 Chair           Furniture            102          2 11-JAN-24      7000
       204 Table           Furniture            102          2 11-JAN-24      7000
       205 Headphones      Electronics          102          2 11-JAN-24      7000
       201 Laptop          Electronics          103          1 15-JAN-24      2000
       202 Mouse           Electronics          103          1 15-JAN-24      2000
       203 Chair           Furniture            103          1 15-JAN-24      2000
       204 Table           Furniture            103          1 15-JAN-24      2000
       205 Headphones      Electronics          103          1 15-JAN-24      2000
       201 Laptop          Electronics          104          3 20-JAN-24      9000
       202 Mouse           Electronics          104          3 20-JAN-24      9000
       203 Chair           Furniture            104          3 20-JAN-24      9000
       204 Table           Furniture            104          3 20-JAN-24      9000
       205 Headphones      Electronics          104          3 20-JAN-24      9000
       201 Laptop          Electronics          105          5 25-JAN-24      3000
       202 Mouse           Electronics          105          5 25-JAN-24      3000
       203 Chair           Furniture            105          5 25-JAN-24      3000
       204 Table           Furniture            105          5 25-JAN-24      3000
       205 Headphones      Electronics          105          5 25-JAN-24      3000
SQL>
SQL> -- Q34: Count total rows produced by cross join.
SQL> SELECT COUNT(*)
  2  FROM CUSTOMER1_24BYB1117
  3  CROSS JOIN PRODUCT_24BYB1117;

  COUNT(*)
==========
        25
```

```
  COUNT(*)
==========
        25
SQL>
SQL> -- Q35: Use WHERE clause to limit cross join output.
SQL> SELECT *
  2  FROM CUSTOMER1_24BYB1117 C
  3  CROSS JOIN ORDERS_24BYB1117 O
  4  WHERE C.CUST_ID = O.CUST_ID;

   CUST_ID CUST_NAME       CITY            ORDER_ID    CUST_ID ORDER_DATE   AMOUNT
========== =============== =============== ======== ========== ============ ======
         1 Arun            Chennai              101          1 10-JAN-24      5000
         2 Divya           Bangalore            102          2 11-JAN-24      7000
         1 Arun            Chennai              103          1 15-JAN-24      2000
         3 Karthik         Hyderabad            104          3 20-JAN-24      9000
         5 Rahul           Mumbai               105          5 25-JAN-24      3000
SQL>
```

36) Cross join produces the Cartesian product of two tables. If table A has m rows and table B has n rows, the

result will contain m × n rows, which can become very large and slow for big datasets.